Drinking Water Potability

SkywaRDS Team Members:

Sarah Alsharif - Renad Alsaidiy - Dareen Aljuhani

College of Computer Science and Engineering —Yanbu

Contents

1- Problem Description.

2- The Importance of Solving the Problem.

3- Important Applications.

4- Description of the Dataset.

5- Overview of the Dataset

6- Dataset Insights

7- Preparing the Dataset

8- Proposed Solution and Results.

9- Model Possible Applications.

Problem Description :

Problems related to the potability of water emerge when it does not meet the WHO standards for the water features such as pH and sulfate, rendering it unsuitable for human consumption. These things is not usually visible to the human eye which makes them dangerous and can give rise to a range of health issues and represent a substantial global public health challenge. Guaranteeing access to pure and safe drinking water is essential to ward off waterborne illnesses and uphold overall health and well-being

The Importance of Solving the Problem:

1. **Human Health:** Access to safe drinking water is fundamental to preventing waterborne diseases, safeguarding public health, and reducing the burden of illness associated with non-potable water consumption.

2. **Basic Human Right:** Clean and safe water is recognized as a basic human right. Addressing water potability ensures that individuals have access to this essential resource without compromising their well-being.

3. **Economic Impact:** Investing in water supply and sanitation improvements can yield net economic benefits. By reducing health care costs and adverse health effects, such interventions contribute to economic development and productivity.

The Importance of Solving the Problem:

4. **Environmental Conservation:** Monitoring water quality parameters contributes to environmental conservation. Protecting water sources and maintaining ecosystems is essential for long-term sustainability and biodiversity.

5. **Global Development Goals:** Addressing water potability aligns with international development goals, such as those outlined by the United Nations, particularly Goal 6: Clean Water and Sanitation.

Our goal is to determine if the water is safe for human consumption based on WHO standards.

Important Applications:

1. **Public Health:** The dataset helps assess the safety of drinking water by examining key quality metrics. Ensuring water meets WHO standards is vital to prevent waterborne diseases and protect public health.

2. **Policy Development:** Findings from the analysis can contribute to the formulation of effective policies related to water supply and sanitation. Understanding water quality trends can guide interventions that yield economic benefits and reduce health care costs.

3. **Resource Allocation:** By identifying areas with water quality issues, resources can be allocated efficiently to improve infrastructure, treatment processes, and regulatory measures where they are most needed.

4. **Preventive Measures:** Early identification of water bodies with poor potability allows for prompt corrective actions, preventing potential health risks and minimizing the adverse effects of contaminated water.

5. **Environmental Impact:** Monitoring parameters like conductivity, turbidity, and organic carbon helps assess the environmental impact of water quality, addressing concerns beyond human health.

6. **Infrastructure Improvement:** Insights from the dataset can guide investments in water supply and sanitation infrastructure, enhancing overall water quality and minimizing risks associated with inadequate systems.

In summary, analyzing this dataset is essential for informed decision-making, safeguarding public health, and implementing measures to ensure access to safe drinking water, aligning with global health standards and promoting sustainable development.

Description of the Dataset:

The dataset that has been used is (Drinking_Water_Potability) which contains detailed information about 3276 water bodies.

The dataset includes vital parameters which are pH levels, Hardness, Total dissolved solids, Chloramines, Sulfate, Conductivity, Organic_carbon, Trihalomethanes and Turbidity.

These features determine whether this water is potable or non-potable. The data provides a comprehensive overview of water quality and helps in making decisions regarding its suitability for human consumption.

Overview of the Dataset

Overview of Data Samples

```
[3] df_water.head()
```

	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity	Organic_carbon	Trihalomethanes	Turbidity	Potability
0	NaN	204.890456	20791.31898	7.300212	368.516441	564.308654	10.379783	86.990970	2.963135	0
1	3.716080	129.422921	18630.05786	6.635246	NaN	592.885359	15.180013	56.329076	4.500656	0
2	8.099124	224.236259	19909.54173	9.275884	NaN	418.606213	16.868637	66.420093	3.055934	0
3	8.316766	214.373394	22018.41744	8.059332	356.886136	363.266516	18.436525	100.341674	4.628771	0
4	9.092223	181.101509	17978.98634	6.546600	310.135738	398.410813	11.558279	31.997993	4.075075	0

```
[4] df_water.tail()
```

	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity	Organic_carbon	Trihalomethanes	Turbidity	Potability
3271	4.668102	193.681736	47580.99160	7.166639	359.948574	526.424171	13.894419	66.687695	4.435821	1
3272	7.808856	193.553212	17329.80216	8.061362	NaN	392.449580	19.903225	NaN	2.798243	1
3273	9.419510	175.762646	33155.57822	7.350233	NaN	432.044783	11.039070	69.845400	3.298875	1
3274	5.126763	230.603758	11983.86938	6.303357	NaN	402.883113	11.168946	77.488213	4.708658	1
3275	7.874671	195.102299	17404.17706	7.509306	NaN	327.459761	16.140368	78.698446	2.309149	1

Data Details Information

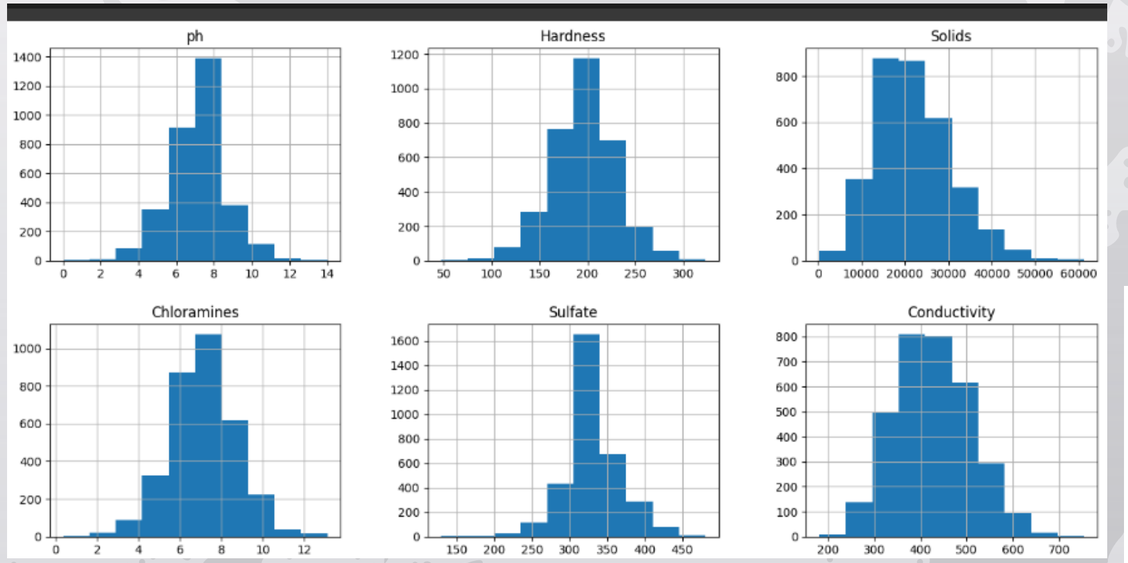
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3276 entries, 0 to 3275
Data columns (total 10 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   ph               2785 non-null   float64
 1   Hardness         3276 non-null   float64
 2   Solids           3276 non-null   float64
 3   Chloramines      3276 non-null   float64
 4   Sulfate          2495 non-null   float64
 5   Conductivity     3276 non-null   float64
 6   Organic_carbon   3276 non-null   float64
 7   Trihalomethanes  3114 non-null   float64
 8   Turbidity        3276 non-null   float64
 9   Potability       3276 non-null   int64
dtypes: float64(9), int64(1)
memory usage: 256.1 KB
```

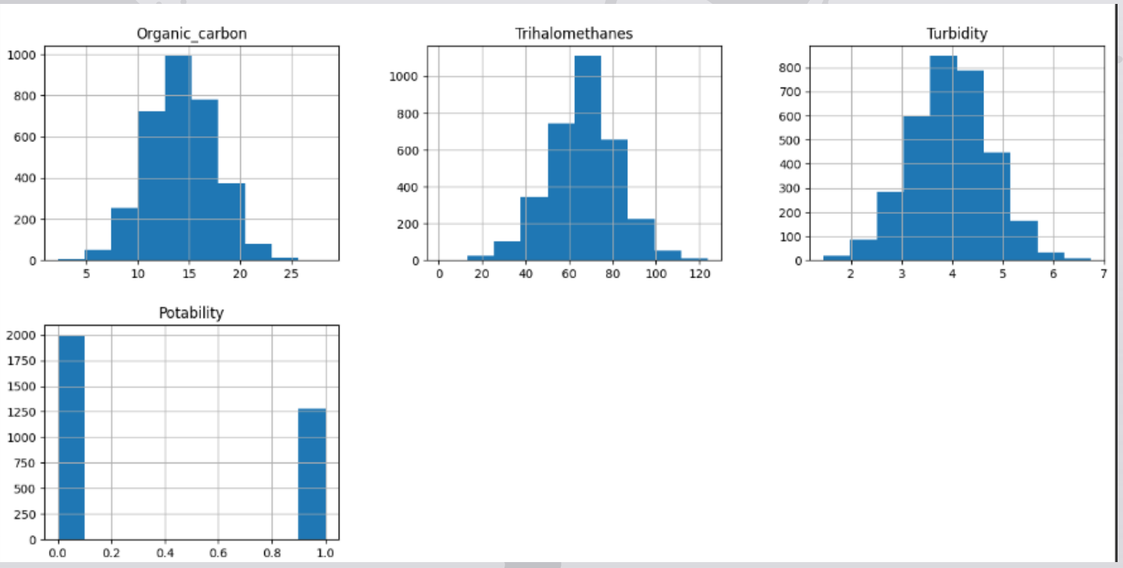
Overview of Data Statistics

```
[6] df_water.describe()
```

	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity	Organic_carbon	Trihalomethanes	Turbidity	Potability
count	2785.000000	3276.000000	3276.000000	3276.000000	2495.000000	3276.000000	3276.000000	3114.000000	3276.000000	3276.000000
mean	7.080795	196.369496	22014.092526	7.122277	333.775777	426.205111	14.284970	66.396293	3.966786	0.390110
std	1.594320	32.879761	8768.570828	1.583085	41.416840	80.824064	3.308162	16.175008	0.780382	0.487849
min	0.000000	47.432000	320.942611	0.352000	129.000000	181.483754	2.200000	0.738000	1.450000	0.000000
25%	6.093092	176.850538	15666.690300	6.127421	307.699498	365.734414	12.065801	55.844536	3.439711	0.000000
50%	7.036752	196.967627	20927.833605	7.130299	333.073546	421.884968	14.218338	66.622485	3.955028	0.000000
75%	8.062066	216.667456	27332.762125	8.114887	359.950170	481.792305	16.557652	77.337473	4.500320	1.000000
max	14.000000	323.124000	61227.196010	13.127000	481.030642	753.342620	28.300000	124.000000	6.739000	1.000000

Data Count and Concentration Histogram





Dataset Insights

College of Computer Science and Engineering — Yanbu

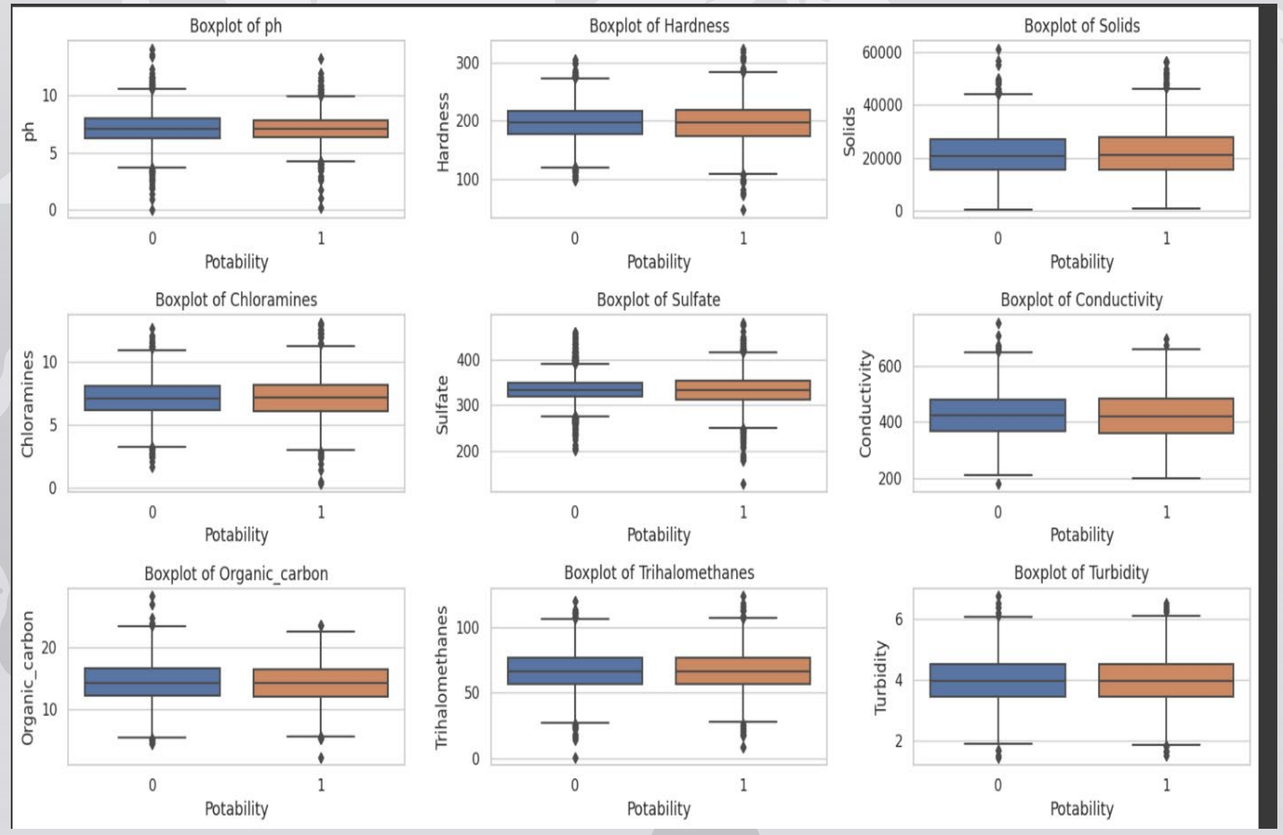
Outliers and Comparison

Outliers

We found few outliers in the dataset, these outliers did not affect the model performance, so it has been left out as its.

Comparison

It has shown that each feature in the dataset has insignificant difference between them if they are potable and when they are non-potable.



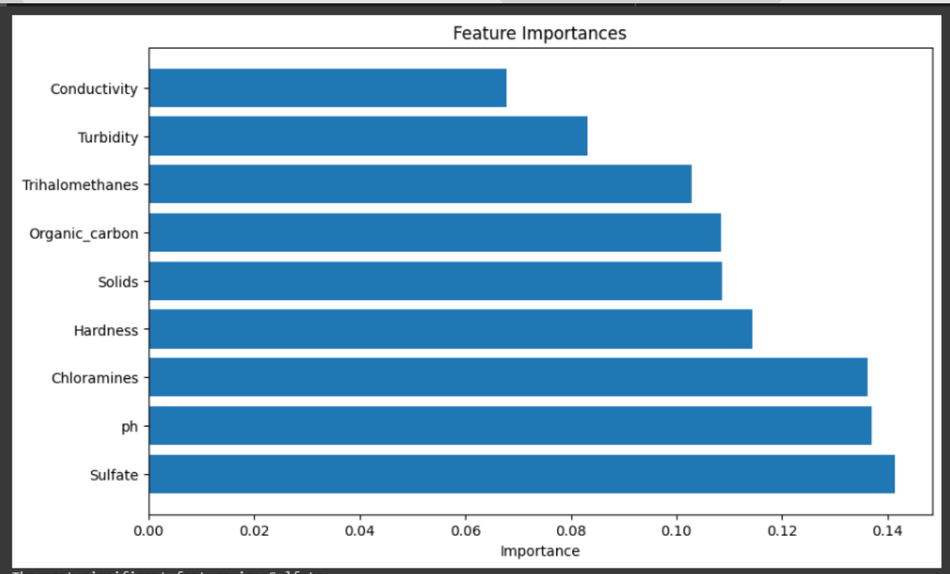
Important Features

An algorithm has been used to identify the most important feature that affect the water potability.

As shown in the graph bar, Sulfate is the most significate feature, right next to it the pH and Chloramines.

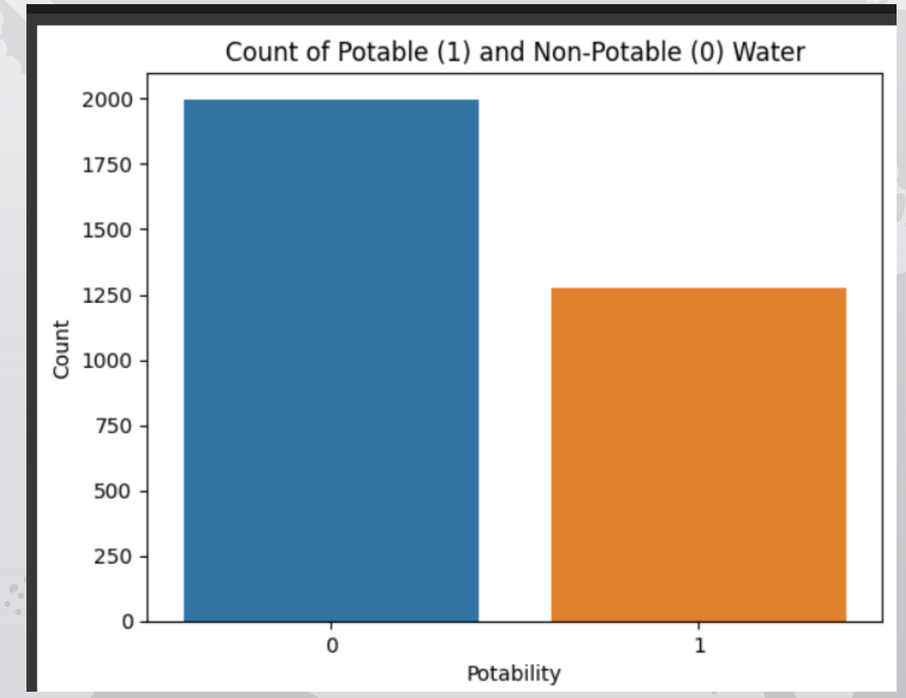
On the other hand, the most insignificant feature is Conductivity.

```
The importance of 'ph' feature is: 12.46%
The importance of 'Hardness' feature is: 12.09%
The importance of 'Solids' feature is: 11.39%
The importance of 'Chloramines' feature is: 11.69%
The importance of 'Sulfate' feature is: 12.58%
The importance of 'Conductivity' feature is: 10.25%
The importance of 'Organic_carbon' feature is: 10.28%
The importance of 'Trihalomethanes' feature is: 9.70%
The importance of 'Turbidity' feature is: 9.56%
```

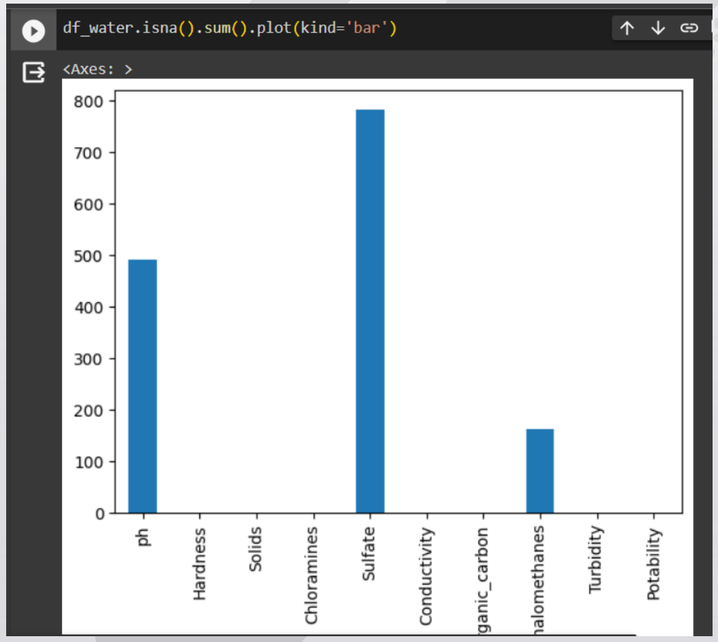



```
The most significant feature is: Sulfate
```

Insights Shows Problems Need To Be Fixed



Imbalanced data

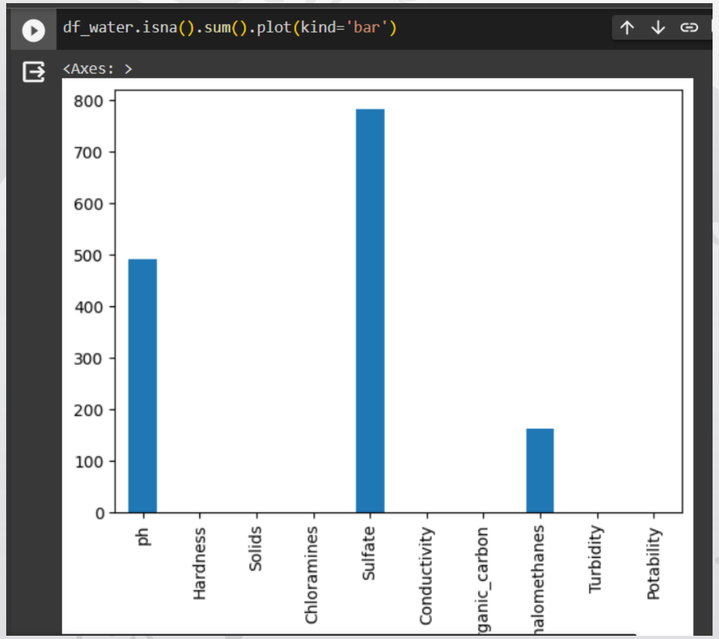


Missing values

Preparing the Dataset

Missing values

There were missing values, and it has been replaced with mean or median values, so the model can work on the dataset without errors.



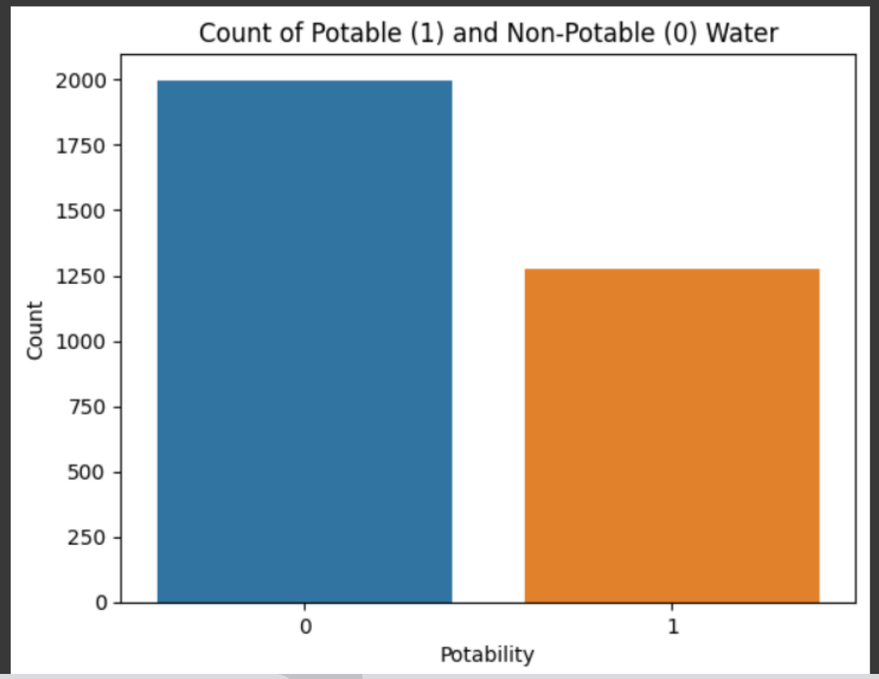

```
[9] df_water['ph'] = df_water['ph'].fillna(df_water['ph'].mean())
    df_water['ph'].isnull().sum()

    sulfate = df_water['Sulfate'].mean()
    df_water['Sulfate']= df_water['Sulfate'].fillna(sulfate)

    df_water['Trihalomethanes'] = df_water['Trihalomethanes'].fillna(df_water['Trihalomethanes'].median())
```

Preparing the Dataset

Imbalanced data

The data was imbalance, it has more samples of non-potable water than the potable one, which needed to be fixed, so the model will not be bias.

```
[ ] # Count plot of Potability
    sns.countplot(x='Potability', data=df_water)
    plt.title('Count of Potable (1) and Non-Potable (0) Water')
    plt.xlabel('Potability')
    plt.ylabel('Count')
    plt.show()
```



Preparing the Dataset

Scaling data

The data has been put in same scale for all data using StandardScaler algorithm, to make it easier for the model to train.

```
import statsmodels.api as sm
import sklearn
from sklearn.preprocessing import StandardScaler
```

```
[ ] X_train, X_test, y_train, y_test = train_test_split(df_water[[c for c in df_water.columns if c != 'Potability']],df_water['Potability'],train_size = 0.7,random_state = 1)
```

```
[ ] sc = StandardScaler()
    X_train[X_train.columns] = sc.fit_transform(X_train)
    X_test[X_test.columns] = sc.transform(X_test)
```

The Proposed Solution and Results

The Proposed Solution

The **`XGBClassifier`** is a class in the **XGBoost library,** a powerful implementation of gradient boosting for supervised learning tasks.

It has the same features as **`GradientBoostingClassifier`**, but it has better performance. Since **`XGBClassifier`** is better than **`GradientBoostingClassifier`**, We chose it as our model.

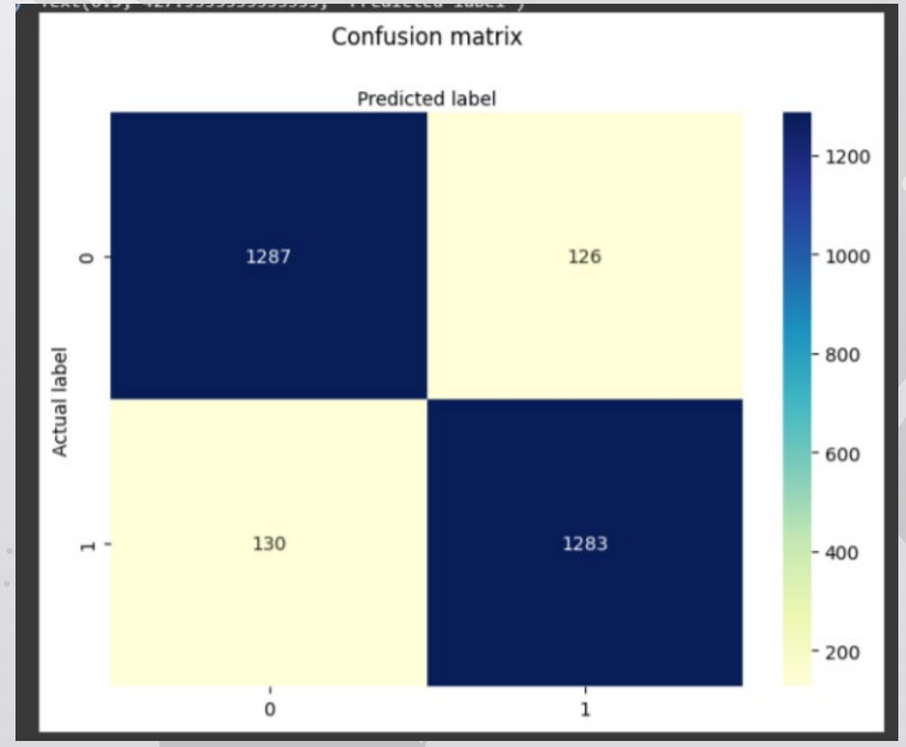
It builds an ensemble of decision trees sequentially, correcting errors with each new tree. Regularization techniques control overfitting, and hyperparameters like `n_estimators`, `learning_rate`, and `max_depth` can be tuned.

The algorithm provides feature importance scores and is known for its strong predictive performance. Example usage involves creating an instance, fitting to training data, making predictions, and evaluating performance metrics like accuracy.

XGBClassifier Model

```
[ ] from xgboost import XGBClassifier
    from sklearn.model_selection import GridSearchCV

    xgb = XGBClassifier(random_state=1)

    param_grid = {
        "max_depth": [3, 5],
        "min_child_weight": [1, 3, 5],
        "subsample": [0.8, 1.0],
        "colsample_bytree": [0.8, 1.0],
        "n_estimators": [50, 100],
        "learning_rate": [0.01, 0.05, 0.1]
    }

    cv = GridSearchCV(estimator=xgb, param_grid=param_grid, cv=5, scoring='balanced_accuracy', verbose=10, n_jobs=-1).fit(X_train, y_train)

    Fitting 5 folds for each of 144 candidates, totalling 720 fits
```

XGBClassifier Model Results:



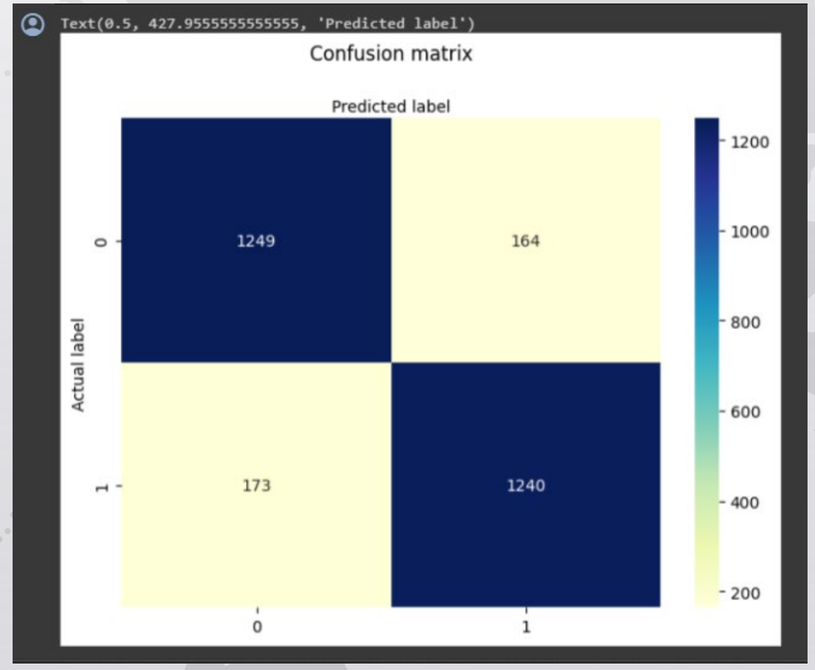

```
[ ] xgb = cv.best_estimator_

    y_train_pred = gb2.predict(X_train)
    cf = confusion_matrix(y_train,y_train_pred)

    print(cf)
    acc = (cf[0,0] + cf[1,1])/np.sum(cf)
    recall = (cf[1,1])/(cf[1,1] + cf[1,0])
    spec = (cf[0,0])/(cf[0,1] + cf[0,0])
    print(f"Accuracy = {acc} Recall = {recall} Specificity = {spec}")

    [[1287  126]
     [ 130 1283]]
    Accuracy = 0.9094125973106865 Recall = 0.9079971691436659 Specificity = 0.910828025477707
```

GradientBoostingClassifier Model

```
gb = GradientBoostingClassifier(random_state=1)
param_grid = {
    "max_depth": [3,5],
    "min_samples_split": [10,20, 50],
    "min_samples_leaf": [30,50],
    "n_estimators": [50,100],
    "learning_rate": [0.01, 0.05, 0.1]
}

cv = GridSearchCV(estimator=gb, param_grid=param_grid,cv=5,scoring='balanced_accuracy',verbose=10,n_jobs=-1).fit(X_train,y_train)
```

```
Fitting 5 folds for each of 72 candidates, totalling 360 fits
```

GradientBoostingClassifier Model Results




```
[ ] gb = cv.best_estimator_

    y_train_pred = gb.predict(X_train)
    cf = confusion_matrix(y_train,y_train_pred)

    print(cf)
    acc = (cf[0,0] + cf[1,1])/np.sum(cf)
    recall = (cf[1,1])/(cf[1,1] + cf[1,0])
    spec = (cf[0,0])/(cf[0,1] + cf[0,0])
    print(f"Accuracy = {acc} Recall = {recall} Specificity = {spec}")

    [[1249  164]
     [ 173 1240]]
    Accuracy = 0.8807501769285209 Recall = 0.8775654635527247 Specificity = 0.883934890304317
```

Model Possible Applications:

Water Management Systems: Integration into water supply and treatment systems to continuously monitor and ensure the potability of water in real-time. This helps in early detection of non-potable water and immediate corrective actions.

Emergency Response: Integration into disaster management systems for rapid assessment of water quality during emergencies, such as natural disasters or industrial accidents, to guide emergency response efforts.

Smart Home Integration: Integration into smart home systems to provide real-time updates on the quality of tap water. This empowers homeowners to make informed decisions about water usage and potentially triggers alerts or actions if water quality deviates from safety standards.

Thank you for listening

DATATHON

DIG DEEP, FIND GOLD

College of Computer Science and Engineering — Yanbu